Controllers in ASP.NET Core Web API

Controllers in ASP.NET Core Web API

In ASP.NET Core Web API, controllers act as the central point for handling client requests and generating responses. They serve as intermediaries between the client and the server, receiving HTTP requests, invoking business logic through services or repositories, and returning data in formats such as JSON or XML.

What are Controllers in ASP.NET Core Web API?

In **ASP.NET Core Web API**, controllers are **special classes** that handle HTTP requests sent by clients (such as browsers, mobile apps, Postman, or other services) and return HTTP responses. Think of them as the "**Traffic Police**" or **Gatekeepers** of your API. They listen to requests at specific endpoints (URLs), process them with the aid of business logic, and return meaningful responses in standard web formats, such as JSON or XML.

In MVC terms:

- **Model** = Data (Entities, DTOs, Domain Models).
- **View** = Not used in Web APIs (unlike MVC web apps).
- **Controller** = The central unit that receives the request, coordinates with services and models, and sends back a response.

Key Roles of a Controller

Let us understand the key Roles and Responsibilities of a Controller in an ASP.NET Core Web API Application.

Request Handling

- The controller listens at a specific URL route (e.g., **/api/products**) and decides which **Action Method** (a public method inside the controller) should handle the request.
- Each action method is decorated with attributes such as **[HttpGet], [HttpPost], [HttpPut], [HttpDelete]** that map to HTTP verbs.

Example: Here, **GET /api/products/5** will be handled by the GetProduct method.

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
```

```
    [HttpGet("{id}")]
    public IActionResult GetProduct(int id)
    {
        // logic to fetch product
        return Ok(new { Id = id, Name = "Laptop" });
    }
}
```

Analogy: Think of a call center where incoming calls (requests) are routed to the correct department (action method).

Data Processing

Controllers **shouldn't hold heavy business logic themselves**. Instead, they delegate:

- To **services** for applying business rules (e.g., price discounts, tax calculation).
- To **repositories** or EF Core for data persistence (CRUD on a database).
- To **validators** for input checking (e.g., ensuring an email format is valid).

They handle input validation, model binding, and error checking before passing data to the services. For example:

```
[HttpPost]
public IActionResult CreateProduct(ProductDto dto)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    var product = _productService.Create(dto);
    return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
}
```

Analogy: The controller is like a **coordinator**. It doesn't do the heavy work but ensures the right team (services, database) gets involved.

Response Generation

After the data is processed, the controller decides **what response to send back**. Responses may include:

- **Response Payload** → Data in JSON/XML format (default is JSON in Web API).
- **Status codes** → To indicate success or failure (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error, etc.).
- **Error messages** → Clear explanation if something went wrong.

Example:

```
[HttpDelete("{id}")]
public IActionResult DeleteProduct(int id)
{
    bool deleted = _productService.Delete(id);
    if (!deleted)
        return NotFound(new { Message = "Product not found" });

    return NoContent(); // 204
}
```

Analogy: Like a waiter in a restaurant, after checking with the chef (services/database), the waiter (controller) delivers the order (response) back to the customer (client).

Why are Controllers Important?

- They **Enforce Separation of Concerns** → Controllers handle HTTP Request, Services handle business logic, repositories handle data access logic.
- They make your API **RESTful** → Each endpoint maps to resources and standard HTTP verbs.
- They improve **maintainability** → Keeping code clean and organized.
- They allow **Flexible Responses** (Status Codes, Payloads, Headers, etc.).

Adding Controller Class in ASP.NET Core Web API

When creating a controller in an ASP.NET Core Web API project, adhere to the following conventions and best practices:

Naming Convention:

ASP.NET Core uses **Conventions** for recognizing controllers. By convention, ASP.NET Core recognizes controller classes whose names end with the "**Controller**" suffix.

- **Must end with Controller suffix**→ This is how ASP.NET Core identifies them during routing.
- Examples:
    - EmployeeController → Manages employees.
    - StudentController → Manages students.
    - ProductController → Manages products.

If you name your class Employee instead of EmployeeController, ASP.NET Core won't recognize it as a controller.

Best Practice:

Choose meaningful names based on the resource you're exposing, not on verbs.

- **Correct Example:** EmployeeController
- **Wrong Example:** ManageEmployeeController or DoEmployeeStuffController

Base Class

Controllers in Web API projects usually inherit from **ControllerBase**.

- **ControllerBase** → Provides features needed for **APIs** (No views, just JSON/XML responses).
- **Controller** → Used in **MVC applications** where you need both API and Razor Views (it inherits from ControllerBase and adds view-related features like View(), PartialView(), etc.).

In Web API, stick with ControllerBase unless you are mixing APIs with Razor views.

```
[ApiController]
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public IActionResult GetEmployees()
    {
        return Ok(new[] { "John", "Jane", "Alice" });
    }
}
```

Recommended Folder Structure:

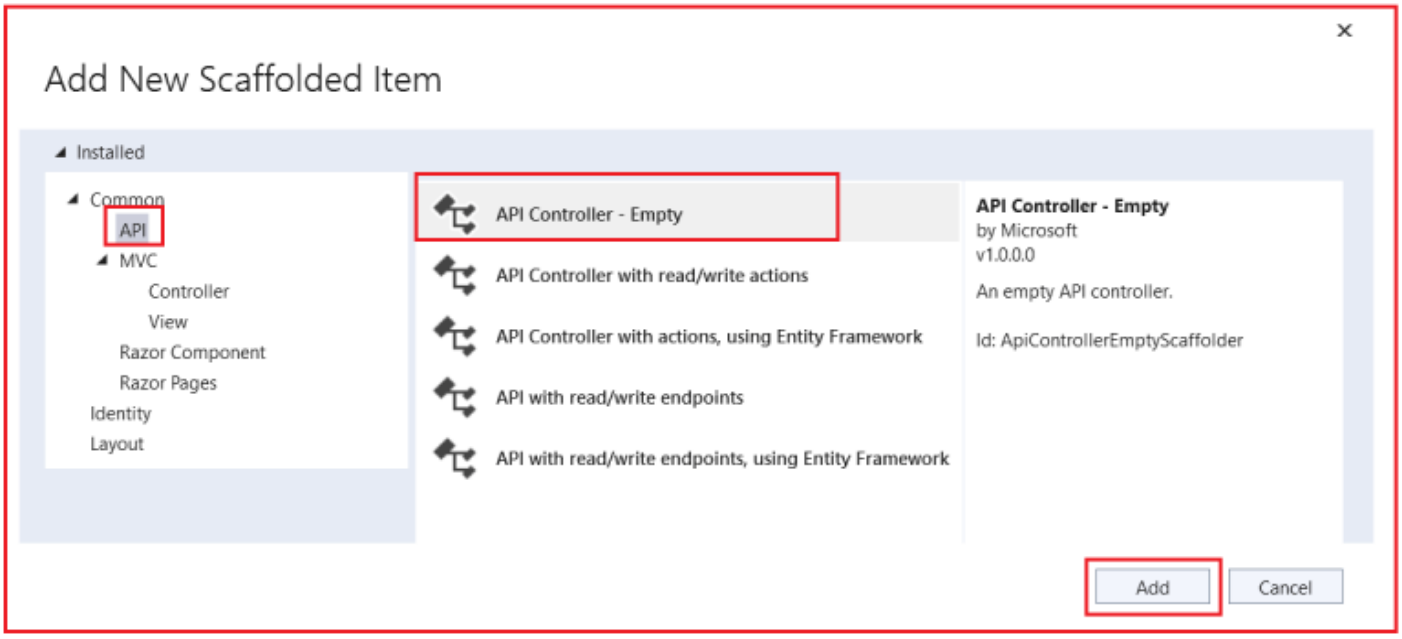
By convention, controllers are placed inside the **Controllers** folder.

- By default, new ASP.NET Core Web API projects have a **Controllers** folder.
- Controllers should always go inside this folder for organization.
- If the folder doesn't exist (e.g., you deleted it), create a folder named Controllers at the root of the project.
- ASP.NET Core does not *require* the folder to be named Controllers, but following the convention makes it easier for teams and tools to locate them.

Why is this important? Keeping controllers in one place makes the project more readable and easier for teams to maintain.

Steps to Add a Controller:

Let's add a controller named Employee. To do so, right-click on the Controllers folder and then select **Add => Controller** from the context menu, which will open the following Add New Scaffold Item window. From this window, please choose **API**, then **API Controller – Empty**, and click on the **Add** button, as shown in the image below.



Understanding the Different Options:

Let us understand the different options for creating an API Controller in ASP.NET Core:

API Controller – Empty

This option provides a blank controller with no predefined actions, allowing you to build everything from scratch. It's ideal for learning, experimenting, and situations where you need maximum flexibility to design custom endpoints and logic without code.

- Contains only class declaration with [ApiController] and [Route].
- No predefined CRUD methods are available; you must add all endpoints manually.
- Best suited for beginners and projects that require full customization.
- Helps in understanding the fundamentals of Routing, HTTP Verbs, and Responses.

Analogy: Like moving into an empty room where you have complete freedom to decide the layout, furniture, and design.

API Controller with Read/Write Actions

This option scaffolds a controller with predefined CRUD (Create, Read, Update, Delete) methods, making it easy to set up a RESTful API quickly. It's ideal for projects where you want a head start and don't want to create standard CRUD endpoints manually.

- Automatically generates methods for Get, Get by ID, Post, Put, and Delete.
- Suitable for quick prototypes or standard APIs that do not require complex logic.
- Reduces repetitive coding and provides a working structure instantly.

- Can be extended later by adding business logic or custom validations.

Analogy: Like moving into a house with basic furniture already in place, you can start living immediately, but you can still customize it later.

API Controller with Actions Using Entity Framework

This option extends CRUD scaffolding by connecting directly to a database using Entity Framework Core. It's perfect when you already have models and a DbContext and want to generate database-ready API endpoints quickly.

- Prebuilt methods that query and update data directly from the database.
- Integrates Entity Framework Core for database interactions.
- Useful for rapid prototyping of database-backed applications.
- Reduces duplicate code for common database operations.

Analogy: Just as with getting a furnished apartment, you can move in and start living right away, but the layout and furniture may not meet your long-term needs.

API with Read/Write Endpoints

This option is similar to the Read/Write Actions template but provides more control over endpoint naming and structure. It's meant for developers who need flexibility in designing endpoints while still benefiting from predefined CRUD scaffolding.

- Generates standard CRUD endpoints but allows custom route names.
- Useful when API design requires adherence to specific naming conventions.
- Balances automation with customization for endpoint structure.
- Suitable for teams following unique API standards.

Analogy: Like buying a furnished house, but with the freedom to choose and arrange the furniture style and placement to your preference.

API Controller with Read/Write Endpoints Using Entity Framework

This option combines the benefits of Entity Framework integration with customizable endpoints, providing a flexible solution. It gives you ready-to-use database CRUD operations while allowing you to rename or extend endpoints beyond the defaults.

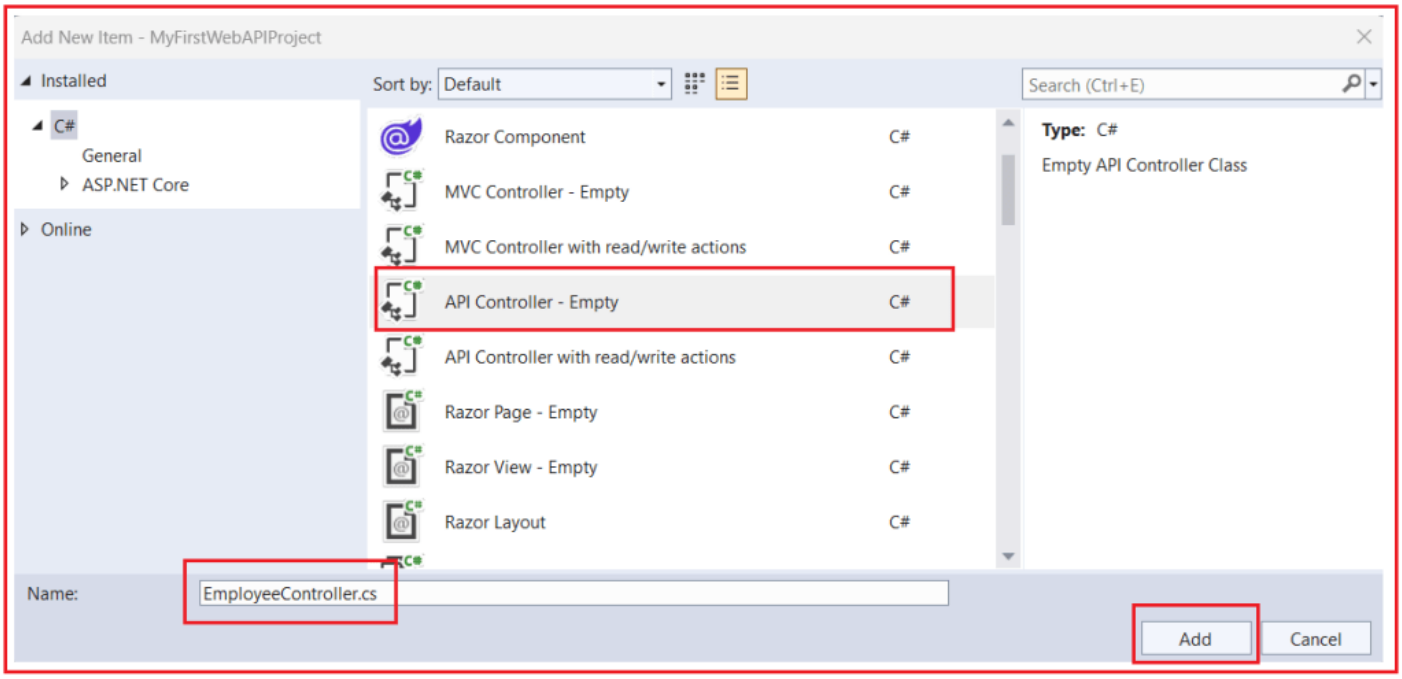
- Direct database integration using Entity Framework Core.
- Allows renaming and customizing routes for more flexibility.

- Ideal for production-ready apps requiring both EF and non-standard endpoint names.
- Saves time by scaffolding EF code while supporting customization.

Analogy: It's like owning a house with a modern kitchen already installed, but you're still free to redesign the interior layout and decorations as you see fit.

Selecting API Controller – Empty

For initial learning and setup, select **API Controller – Empty** to understand the fundamentals before exploring more advanced templates. Therefore, please choose **API Controller – Empty** and click the Add button. This will open the "Add New Item" window. Here, you need to give the controller the name EmployeeController and then click on the Add button, as shown in the image below.



Once you click on the **Add** button, it will add the **EmployeeController** within the **Controllers** folder with the following code. As you can see, the Employee Controller is created without any additional logic. I mean, there are no actions defined within the Controller, and this is because we chose **'API Controller – Empty'** when creating the controller.

```
using Microsoft.AspNetCore.Mvc;
namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]/[action]")]
    [ApiController]
    0 references
    public class EmployeeController : ControllerBase
    {

    }
}
```

As you can see in the above code, in **ASP.NET Core Web API**, a **Controller** is a **C# Class** that inherits from **ControllerBase** and is decorated with the **[ApiController]** attribute.

Modifying the EmployeeController Class:

Let us add one action method within the Employee Controller to return a simple string. So, please modify the EmployeeController class as shown in the code below.

```
using Microsoft.AspNetCore.Mvc;
namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public string Get()
        {
            return "Returning from EmployeeController Get Method";
        }
    }
}
```

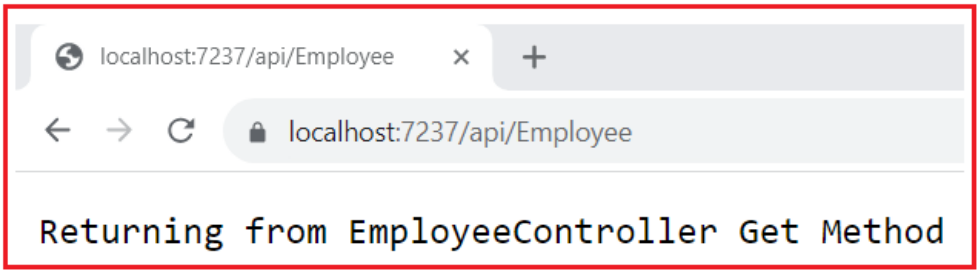
Code Explanation:

The EmployeeController class is a simple ASP.NET Core Web API controller that defines an endpoint to handle HTTP GET requests. It uses routing attributes to determine how clients access it, the [ApiController] attribute to enable Web API-specific features, and inherits from ControllerBase to provide helpful response methods without the overhead of view support. This setup is the foundation of most Web API controllers, ensuring clean routing, validation, and lightweight response handling.

- **[Route("api/[controller]")]** → Sets the base route; [controller] is replaced with the controller name (employee), making the URL /api/employee.
- **[ApiController]** → Marks the class as an API controller, enabling automatic model binding, validation, and cleaner error handling.
- **EmployeeController : ControllerBase** → Inherits from ControllerBase (not Controller), which is designed for Web APIs without view rendering.
- **[HttpGet]** → Maps the Get() method to HTTP GET requests, so a GET request to /api/employee will trigger this method.
- **Get() Method** → A simple action method that currently returns a string response; in real-world apps, this would typically return employee data from a database.

Analogy: Think of the controller as a receptionist in an office. When someone (the client) comes in and asks a question (GET request), the receptionist looks at the request type and provides the appropriate response, in this case, a simple message string.

With the above changes in place, now run the application and navigate to /**api/employee** URL, and you should see the following result.



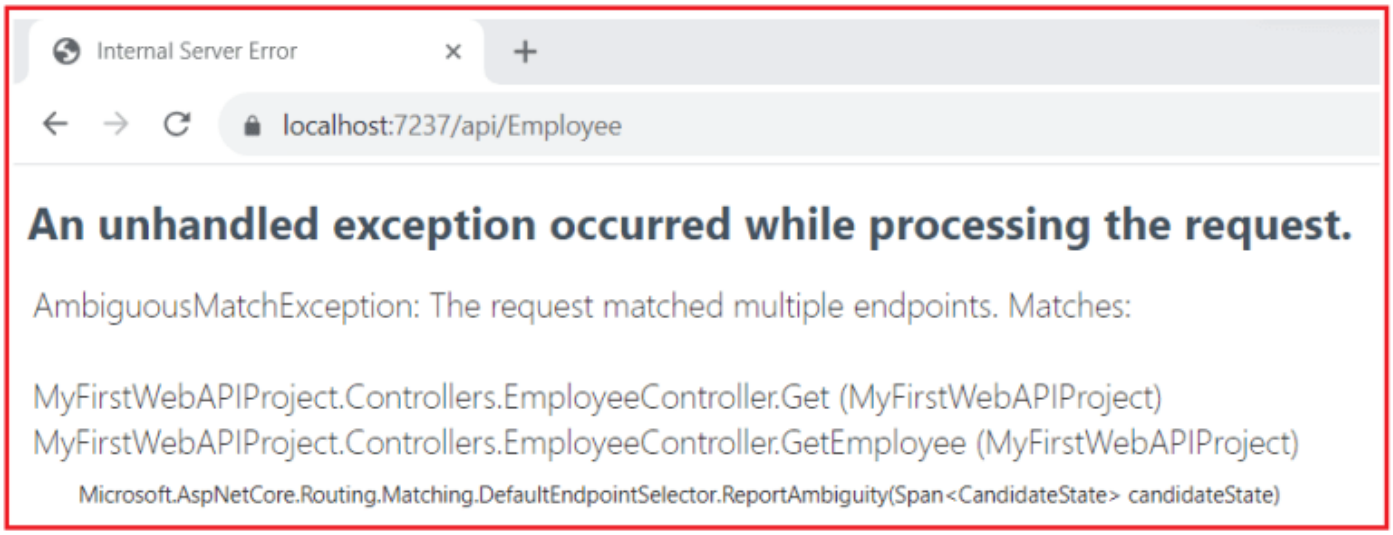
Adding Another Action Method:

Let's add another action method to the Employee Controller class as follows. Now, the controller has two action methods, both of which are decorated with the HttpGet attribute, meaning they can respond only to GET HTTP requests.

```
using Microsoft.AspNetCore.Mvc;
namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public string Get()
        {
            return "Returning from EmployeeController Get Method";
        }
```

```
        [HttpGet]
        public string GetEmployee()
        {
            return "Returning from EmployeeController GetEmployee Method";
        }
    }
}
```

With the above changes, run the application and navigate to the **/api/employee** URL. You will then encounter the following exception.



Why Do We Get the Ambiguous Match Exception?

When both Get() and GetEmployee() methods are decorated with [HttpGet] and use the same base route (api/employee), the ASP.NET Core routing system cannot decide which action method should handle the request. This creates **ambiguity** because both methods are mapped to the same URI. Since the framework doesn't know which one to execute, it throws an **Ambiguous Match Exception** at runtime.

- Both methods are decorated with [HttpGet], meaning both respond to GET requests.
- The controller's base route is [Route("api/[controller]")], which translates to api/employee.
- The framework tries to match a GET request to /api/employee, but finds **two valid matches**.
- Since the runtime cannot decide which one to invoke, it throws an **AmbiguousMatchException**.
- In RESTful design, every endpoint must have a **unique URI** to identify a resource unambiguously.

Analogy: Imagine having two doors in a building with the same number "101." When someone tries to enter room 101, they get confused. Which door should they open? The system crashes because it can't resolve the conflict.

Resolving Route Ambiguity:

As per RESTful Service, each resource should have a Unique Identifier. Let us make some changes to our Route Attribute so that each request has a unique URI. Please modify the **EmployeeController** as follows.

```
using Microsoft.AspNetCore.Mvc;
namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]/[action]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        public string Get()
        {
            return "Returning from EmployeeController Get Method";
        }

        [HttpGet]
        public string GetEmployee()
        {
            return "Returning from EmployeeController GetEmployee Method";
        }
    }
}
```

Code Explanation:

To fix the ambiguity, we make the **Route More Specific** by including the action method name in the URI. This is done by updating the route template to **[Route("api/[controller]/[action]")]**. Now, each method maps to a **Unique Endpoint**: /api/employee/get for Get() and /api/employee/getemployee for GetEmployee(). This ensures clarity and eliminates conflicts in request handling.

- [Route("api/[controller]/[action]")] → Adds the action name to the URL, creating unique routes.
- The Get() method is now accessible at **api/employee/get**.
- The GetEmployee() method is now accessible at **api/employee/getemployee**.
- This resolves ambiguity since each HTTP GET request has **only one clear match**.

Analogy: This is like giving each door in the building a **different number** (101 and 102). Now, when someone looks for a specific room, there's no confusion; the address uniquely identifies the destination.

Note: Here, I have provided an introduction to Controllers, and in our upcoming sessions, we will discuss how to implement the GET, POST, PUT, PATCH, and DELETE Methods in a proper manner.

Difference Between the Controller and ControllerBase Class in ASP.NET Core:

In ASP.NET Core, both the Controller and ControllerBase classes serve as base classes for controllers; however, they are designed for different scenarios and provide distinct sets of functionalities and features.

ControllerBase Class

The ControllerBase class is a **lightweight base class** explicitly intended for building Web APIs that return data rather than views. It provides all the fundamental features needed for handling HTTP requests and responses while intentionally excluding view rendering functionalities. This makes it faster and more efficient for developing RESTful APIs.

- Provides **core MVC features, including** routing, action methods, model binding, and access to the HTTP context.
- Does **not include view rendering support** (no View(), PartialView(), ViewBag, or ViewData).
- Provides **helper methods** such as Ok(), NotFound(), BadRequest(), and CreatedAtAction(), among others, for standard HTTP responses.
- Designed to be **lightweight**, avoiding the overhead of server-side rendering.
- Best suited for **RESTful APIs** returning JSON or XML responses.

Analogy: Think of ControllerBase like a delivery service that only handles **packages** (data) and does not serve dine-in customers. It's focused, efficient, and only does one job — delivering data.

Controller Class

The Controller class extends ControllerBase and provides **support for view rendering**, making it an ideal choice for traditional MVC applications. It not only handles API-style requests but also provides tools for serving server-rendered HTML views using Razor.

- Inherits everything from ControllerBase (all core API features).
- Adds **view-related features**, such as View(), PartialView(), ViewBag, and ViewData.
- Enables rendering **Razor views** (server-side HTML templates).
- Useful when controllers need to handle both **data operations** and **rendering HTML pages**.
- Best suited for **MVC web applications** that incorporate server-side rendering as part of their architecture.

Analogy: Think of Controller like a **full-service restaurant** that not only provides delivery (data responses) but also offers dine-in with waiters, menus, and ambiance (view rendering). It does more, but it also incurs additional overhead.

For a better understanding, please look at the following image:

Feature	ControllerBase	Controller
View Support	❌ Does **not** support view rendering (no Razor views).	✅ Supports view rendering (e.g., Razor views).
Helper Methods	✔ Provides methods like `Ok()`, `NotFound()`, `BadRequest()`, etc.	✔ Inherits all `ControllerBase` helper methods. ✅ Adds view-related methods like `View()`, `PartialView()`, etc.
View Data Access	❌ No access to `ViewBag`, `ViewData`, or `TempData`.	✅ Access to `ViewBag`, `ViewData`, and `TempData`.
Model Binding and Validation	✔ Supports model binding and validation for APIs.	✔ Supports model binding and validation for APIs and MVC views.
Routing	✔ Uses attribute routing by default.	✔ Uses attribute routing by default. Additionally supports conventional routing for MVC.
Response Types	Primarily returns data (e.g., JSON, XML).	Can return data and render views.

Choosing Between Controller and ControllerBase

When creating a new controller, the choice depends entirely on the type of application you're building.

Use ControllerBase when:

- You are building a **Web API** focused on returning JSON/XML responses.
- You don't need server-side rendered views.
- You want a **lightweight, optimized** setup for data-driven endpoints.

Use Controller when:

- You are building a **traditional MVC application** that requires rendering Razor views.
- Your controller must handle both **data endpoints and HTML views**.
- You plan to use **ViewBag, ViewData, and Razor syntax** for rendering UI.

Analogy: Choosing between ControllerBase and Controller is like choosing between a **food delivery service** and a **full-service restaurant**. If you only need fast, efficient delivery of food (data), choose delivery (ControllerBase). If you also want the dining experience with waiters and decor (view rendering), choose the restaurant (Controller).

For a better understanding, please look at the following image:

Scenario	Use `ControllerBase`	Use `Controller`
Building a RESTful API	✅ Ideal choice for API endpoints without views.	❌ Not recommended if views are not needed.
Developing an MVC Application with Views	❌ Lacks support for rendering views.	✅ Necessary for rendering HTML/Razor views.
Creating Lightweight APIs	✅ More lightweight as it excludes view support.	❌ Includes unnecessary view-related overhead.
Combining API Endpoints with Server-Side Views	❌ Best kept separate to maintain clear responsibilities.	✅ Allows mixing API endpoints with view-rendering actions.

Controllers are the backbone of ASP.NET Core Web API applications, ensuring smooth communication between clients and servers. By following proper naming conventions, applying routing strategies, and choosing the right base class (ControllerBase for APIs or Controller for MVC with views), developers can create scalable and efficient applications. Ultimately, controllers provide a structured way to handle requests, execute logic, and deliver consistent responses, making them an essential building block of modern web APIs.

DOT NET TUTORIALS

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

4 thoughts on “Controllers in ASP.NET Core Web API”

ANURAG SINGH PARIHAR
NOVEMBER 22, 2024 AT 12:11 AM

Hi
Can you please give next button at every page of the end so that we need not to scroll up and then we go to next page

Reply

YI YI PO
NOVEMBER 29, 2024 AT 1:31 PM

Anurag singh parihar,
We are on the same page; I would like to request the next button at the end of every pages, It is more userfriendly.

Reply

SHARATH
JANUARY 18, 2025 AT 8:54 PM

This website is really very helpful for Dotnet Developers who want to learn the technology. It is presented in very good manner. Thank you sir for providing this in-depth explanation.

Reply

SAMEERA
JANUARY 31, 2025 AT 1:42 PM

Thank for your best explanation.

Reply